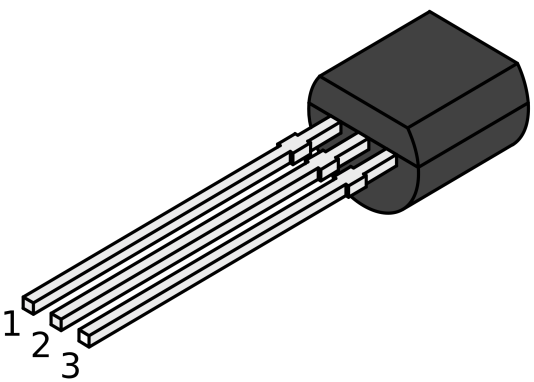
Välkomna tillbaka!

Hur har det gått med er kompilator?

(Hur gick ~~basmatten~~ diskmatten?)

Processorer

Elektronik & logik



Local Man Invents New Device In Electronics For Bell Lab; Could Revolutionalize Radio

William Shockley et al, Bell Labs 1947



The Madison Eagle

Official Newspaper for Chatham Township, Florham Park and Madison

www.RFCafe.com
Page retrieved using paid subscription to Newspapers.com

VOL. LXVI, NO. 27 — MADISON, MORRIS COUNTY, NEW JERSEY, THURSDAY, JULY 1, 1948 — PRICE SEVEN CENTS

Mainly about Madison

Man Found Dead, Wedged in Drain On Park Estate

Ignore Madison Store in Buying Boro Equipment

Marshall Named Bank Director

Legion is Losing On Lung Drive

Invention Replaces Vacuum Tube

Local Man Invents New Device In Electronics For Bell Lab; Could Revolutionize Radio

Madison Scientist Directs Development Of Diminutive Instrument Performing Function of Present-day Vacuum Tube

Holloway Takes Office Today As Drew Head

Madison to Mark Fourth of July At Big Program

Work of Convention Done In Private; Public Got Oratory

Lawyer Fined $50 On Zone Charge

Reverend Haag to Leave Boro; Transferred to Sussex Parish

Mayor, Council Back Pet Show

Oppose Officials At Flood Meeting

Ex-Drew Head, Successor Confer

Signs of the Time Delayed - No Posts

Sorry, But We Must!

First National Dividend Up

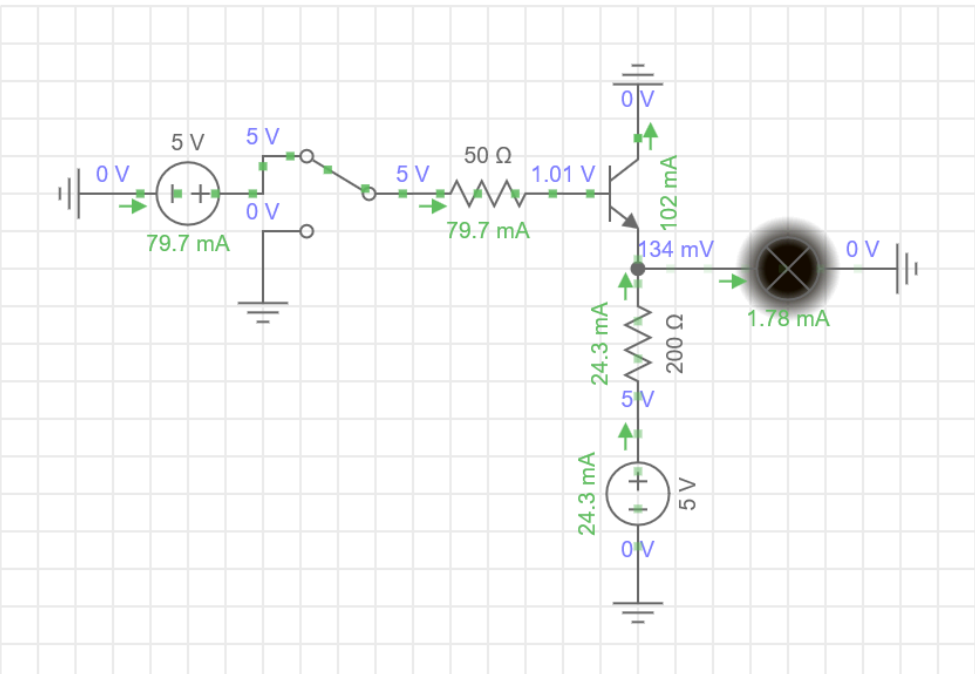
Urges Fleed Veto

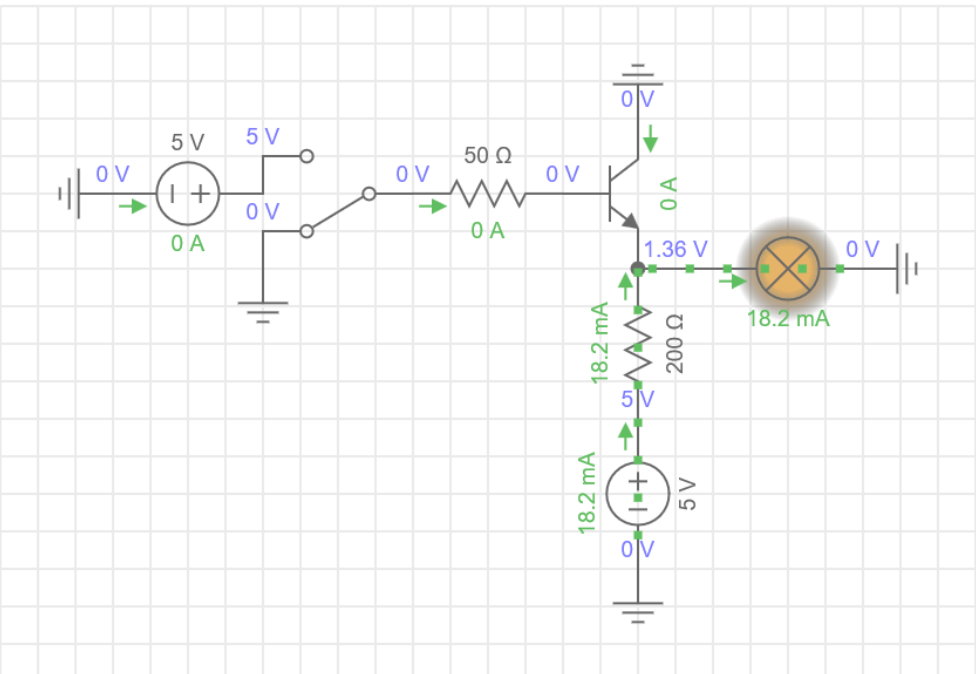
Hear Ye! Hear Ye!

Vito Sylvestre Dies Held High Esteem

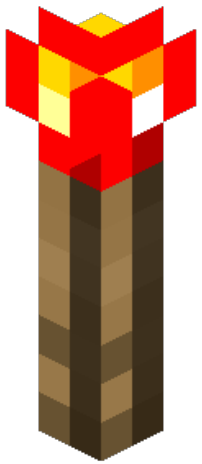
Births

Transistorn kan betraktas som en spänningskontrollerad strömbrytare

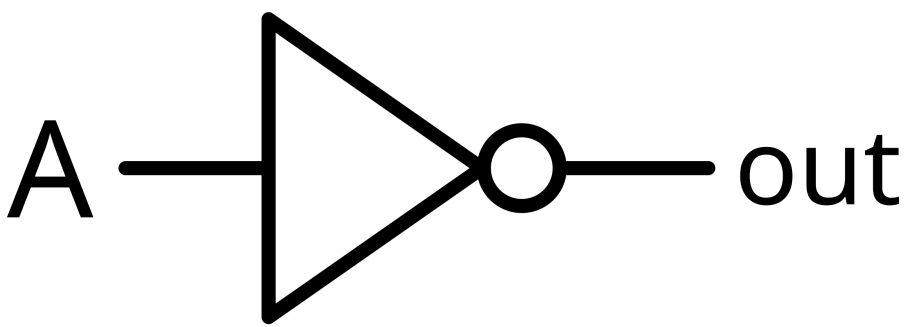




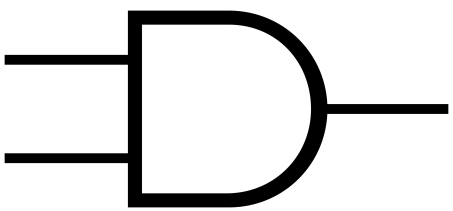
Från elektronik till logik



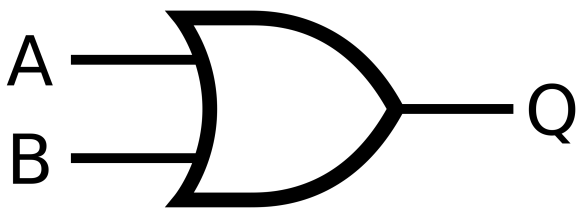




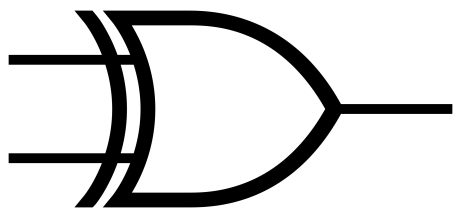
Transistorer sätts ihop för att bygga logiska grindar



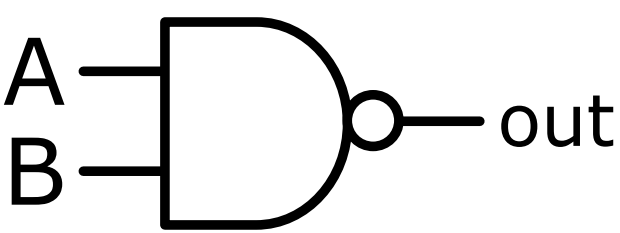
AND-grind



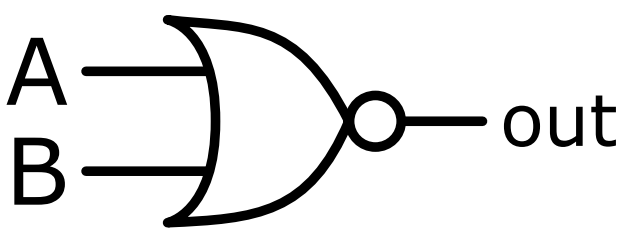
OR-grind



XOR-grind

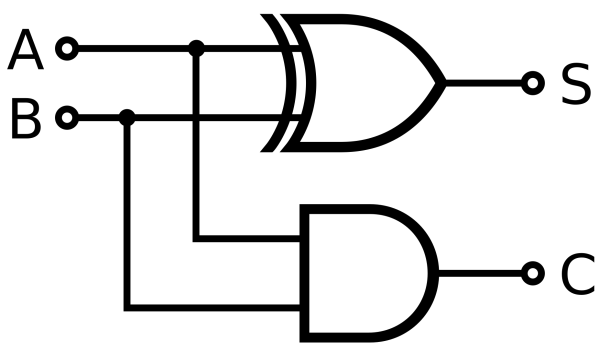


NAND-grind

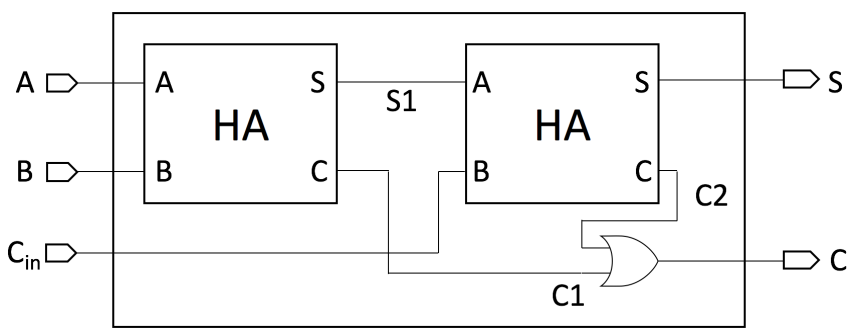


NOR-grind

Logiska grindar sätts ihop för att göra mer komplicerad logik och aritmetik



Halvadderare (Half adder circuit)



Heladderare (Full adder circuit) byggd av två halvadderare

Processorarkitekturer

ISA (Instruction Set Architecture)

- En abstrakt modell av processorer
- Definierar gränssnittet på programmeringsnivå
- Registeruppsättning och -storlek
- Instruktionsuppsättning och -kodning
- Virtuellt minne
- Exempel: **x86**, **ARM**, **MIPS**, **RISC-V**

Register

- Små och snabba minnesenheter i processorn
- General purpose registers - används för godtyckliga beräkningar
- Program counter - addressen för nästa instruktion
- Stack pointer - addressen för toppen på stacken

Assembly

- Lågnivåspråk som mappas direkt till maskinkod
- Olika för olika arkitekturer

```
_start:
        mv   s0, ra

        li   a0, 0
        la   a1, 0
        jal main

        mv   ra, s0
        jr   ra
```

- **CISC** (Complex Instruction Set Computer)
- Komplexa instruktioner som utför flera lågnivåoperationer
- Kan exempelvis läsa minne, göra aritmetik och skriva minne med en instruktion
- T.ex. **x86**

```
addl $0x12345678, (%rax)
```

- Vanligt med **variable length encoding**
- Mindre storlek på program

```
$ objdump -d add
...
   0: 81 00 78 56 34 12     addl   $0x12345678,(%rax)
```

- **RISC** (Reduced Instruction Set Computer)
- Samma operation kräver fler, enklare instruktioner
- Plats för fler register
- T.ex. **RISC-V**

```
addi t1, zero, 0x678
lui  t1, 0x12345
lw   t0, 0(ra)
add  t0, t0, t1
sw   t0, 0(ra)
```

- Vanligt med samma storlek på instruktioner
- Förenklar fetch, decode etc.

```
$ riscv64-none-elf-objdump -d add
...
   0: 67800313                li  t1,1656
   4: 12345337                lui t1,0x12345
   8: 0000a283                lw  t0,0(ra)
   c: 006282b3                add t0,t0,t1
  10: 0050a023                sw  t0,0(ra)
```

Privilegier

- En ISA kan definiera olika ”privilegienivåer” som processorn kan operera i
- x86: *protection ring*, ARM: *exception level*
- Användarprocesser körs i lägsta nivån
- Operativsystemkärna och drivrutiner i en högre nivå
- Tillgång till resurser som minne, filer, nätverk etc. sker vi *systemanrop*

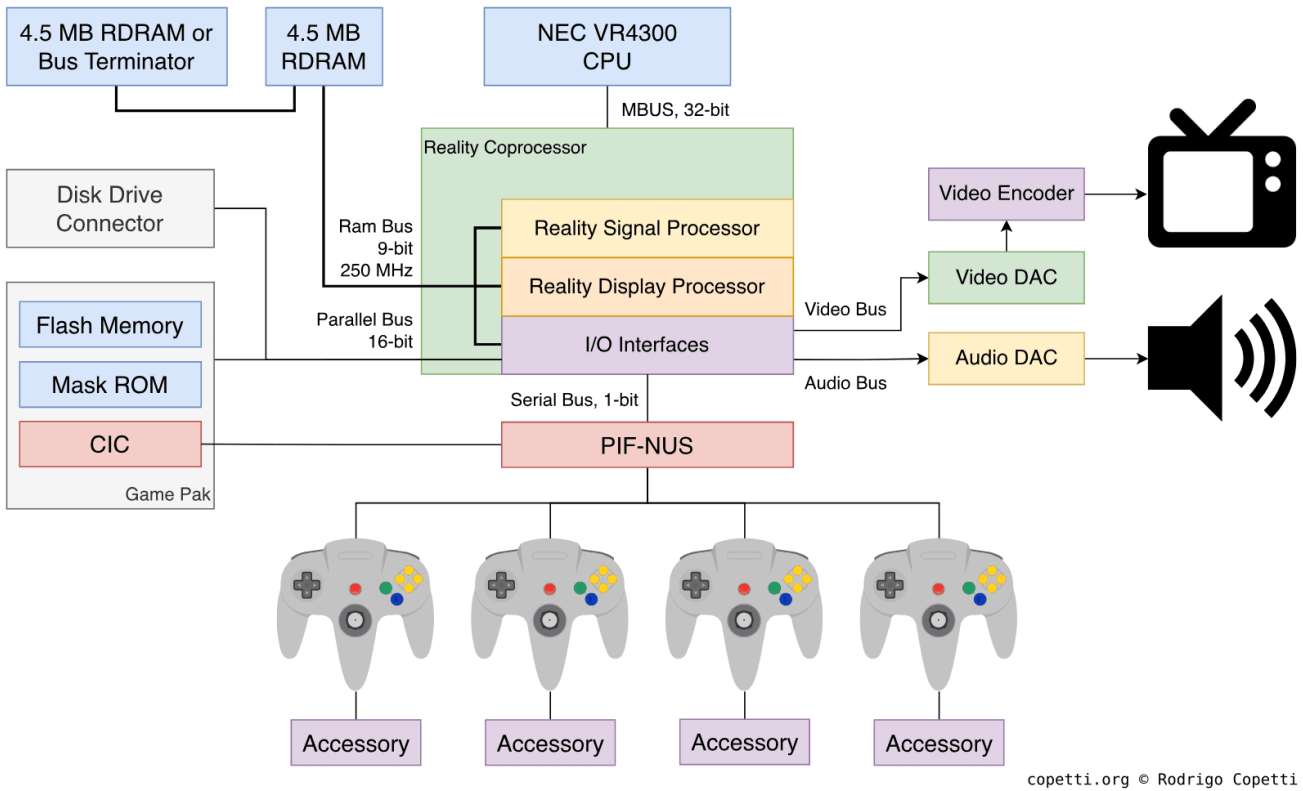
Virtuellt minne

- Virtuella addresser översätts till fysiska addresser på hårdvarunivå
- Sker via en tabell som operativsystemet sköter
- Ser till att användarprocesser är isolerade
- Operativsystemet kan ge en kontinuerlig minnesrymd till processer
- Minne kan även skrivas till disken (swapping)

Virtuellt minne

```
size_t page_size = sysconf(_SC_PAGESIZE);
void *ptr = mmap(NULL,
                 page_size,
                 PROT_READ | PROT_WRITE,
                 MAP_PRIVATE | MAP_ANONYMOUS,
                 -1, 0);
```

Emulering



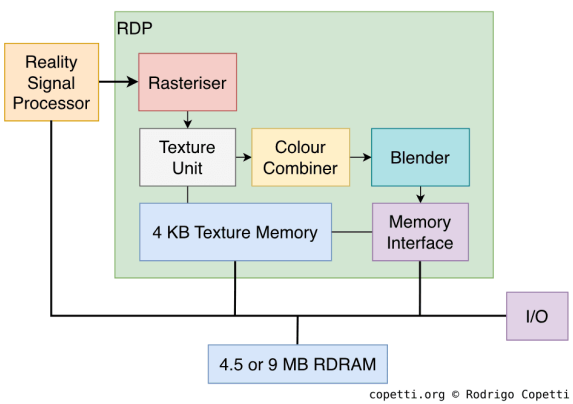
Arkitekturdiagram för Nintendo 64

Spelkonsolen Nintendo 64 har flera olika komponenter som lämpar sig för olika typer av emulering

Hardware Emulation

- Simulerar hårdvarukretsar
- Hög precision
- Ofta komplext och långsamt

Kan vara simulering av en krets som beskrivs av ett Hardware Description Language, exempelvis Verilog eller VHDL.



N64's RDP är en ASIC som kan emuleras väl med Hardware Emulation

```
module toplevel(clock,reset);
input clock;
input reset;

reg flop1;
reg flop2;

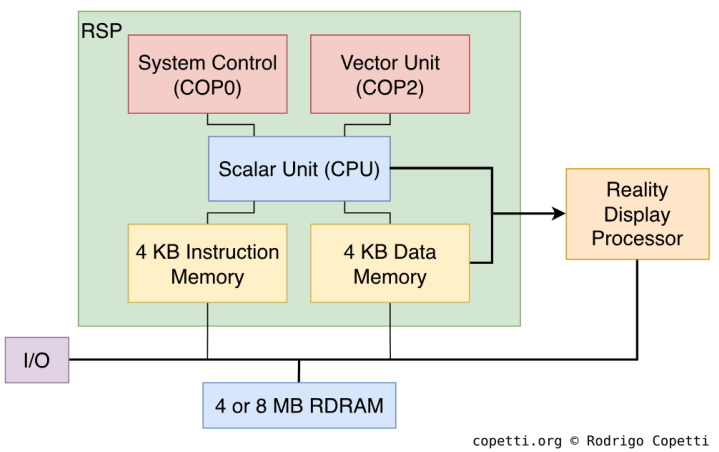
always @ (posedge reset or posedge clock)
  if (reset)
    begin
      flop1 <= 0;
      flop2 <= 1;
    end
  else
    begin
      flop1 <= flop2;
      flop2 <= flop1;
    end
endmodule
```

Exempel av ett Verilog-program från wikipedia

Software Emulation

- *Interpreter* är den enklaste formen av mjukvaruemulering
- En undertyp av software emulation som ibland kallas *Low Level Emulation* (LLE)
- **State** som motsvarar processorns ISA
- **Läser** och **Simulerar** en instruktion i taget och uppdaterar state
- Balans av precision och prestanda

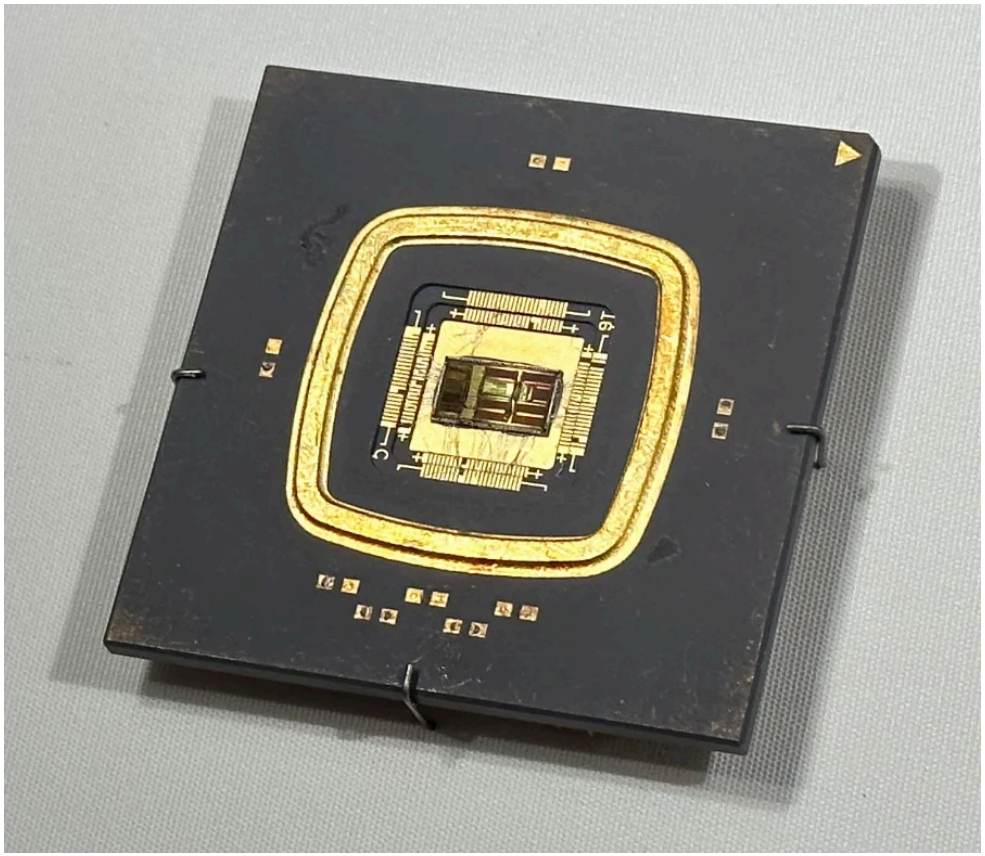
```
state = {
  pc = 0,
  gp = {},
}
while true do
  read_next_instruction()
  decode_instruction()
  execute_instruction()
```



N64's RSP är en programmerbar MIPS-koprocessor som kan emuleras med Software Emulation

Software Emulation

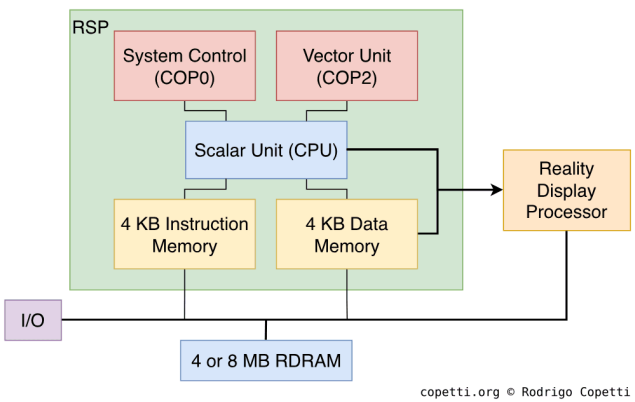
- *Recompiler* (ibland synonymt med JIT)
- **Översätter** programmet till maskinkod som kan köras direkt på värdmaskinen
- **Bättre prestanda** än interpreter
- Kräver dock ofta kompromisser av precision
- Fler utmaningar för komplicerade ISA:er
  - Virtuellt minne och adressrymd
  - Kodmodifieringar
  - Exceptions
  - Cache
  - Timing

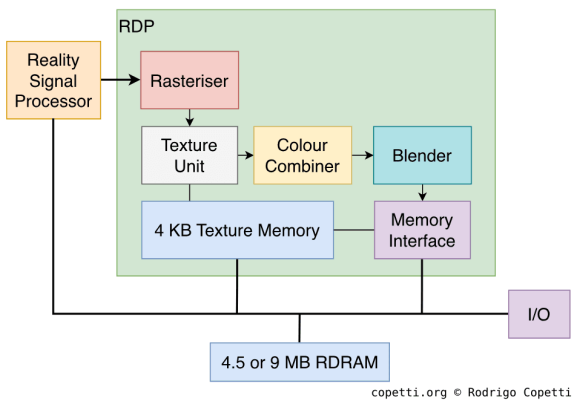


N64's MIPS r4300i CPU lämpar sig väl för både emulering med interpreter och recompiler. Ofta används en blandning av båda metoder.

Software Emulation

- High Level Emulation (HLE)
- Simulerar **effekterna** av ett program istället för att utföra **stegen**
- Exempel
  - Systemanrop
  - Gränssnitt till applikations-specifika kretsar
  - Förprogrammerade koprocessorer eller FPGA:er





RSP är ofta programmerad med ett av ett fåtal väl kända program, då fungerar HLE bra.

RDP är en förhållandevis enkel krets som inte är programmerbar, även här fungerar HLE ofta lika bra som Hardware Emulation.

Läxa

Skriv en emulator för en existerande eller påhittad arkitektur!

- Förslag:
  - Chip8
  - Nand2Tetris
  - 8080 (föregångare till x86)
- Grupper på 2-3 personer

Vad behövs för att emulera ett system?

- Datastruktur för systemets tillstånd

```
struct cpu {
  uint16_t    pc;
  uint8_t     registers[16];
};
struct system {
  struct cpu  cpu;
  uint8_t     memory[1024 * 16];
};
```

Vad behövs för att emulera ett system?

- Avkodning och simulering av instruktioner

```
uint8_t opcode = system.memory[system.cpu.pc++];
if (opcode == OPCODE_ADD) {
  uint8_t operands = system.memory[system.cpu.pc++]
  int reg_a = (operands >> 4) & 0xF;
  int reg_b = (operands >> 0) & 0xF
  system.cpu.regs[reg_a] += system.cpu.regs[reg_b];
} else
...
```